EviRecall

Evidence-Aware Memory for Long-Horizon Multi-Agent Tasks

Project proposal v0.2 | 27 October 2026 | CSC5051 / CSC6052 / MDS5110

Team: Member A, Member B, Member C (names and student IDs to be added). **Status:** proposed research; Smurfs is selected, while dataset scope and the intervention will be frozen after a feasibility pilot. No end-to-end performance results are claimed.

1 Motivation and research questions

Long-running agents reuse compressed histories, tool observations, and conclusions produced by other roles. A summary can omit a decisive constraint; an outdated but well-cited conclusion can then influence later decisions. For example, a planner may reuse a travel plan after the user changes a budget, while a verifier checks only the final answer. This illustrates a potential failure, not an observed result.

We will study this problem in Smurfs [1]. **RQ1:** How does tiered memory affect recall and downstream task completion? **RQ2:** Does checking shared evidence validity reduce stale-information reuse and improve recovery? **RQ3:** How does this trade-off change with retrieval budgets and service load? Resource experiments evaluate the same system; they are not a separate GPU-scheduling project.

2 Related work and technology selection

Agent backbone. Smurfs provides explicit planning, execution, answering, and verification roles and connects to the instructor's research [1]. Its tool-planning results do not establish persistent shared memory or long-horizon performance. Deep Agents offers subagents and context management [8]; Magentic-One targets broader web/file workflows [11]. They remain alternatives rather than additional implementation targets. TwinMarket and EasyMED introduce financial and medical evaluation requirements outside our scope [12] [13].

Memory backend. TierMem links compressed memory to raw evidence and allocates retrieval effort between the two tiers [2]. Its repository also provides lightweight Jev TierMem [3]. We will first test Jev's Python interface, avoiding reproduction of the paper's trained router and vector-service stack. The trade-off is an additional judgment-service dependency and integration work. Jev and the trained paper implementation must be identified separately. A-MEM and Mem0 inform our comparison of memory linking and updating, but neither is a required implementation target [14] [15].

Closest work and contribution. STALE provides both an implicit-conflict benchmark and CUP-Mem, which already performs state consolidation and propagation-aware updates [10]. Governed Shared Memory already studies temporal supersession and cross-agent provenance [7]. We therefore do not claim invalidation, provenance links, or shared memory as new ideas. Our intended contribution is a reproducible Smurfs integration and a controlled study of when evidence validation prevents cross-role error propagation under limited budgets. Stronger novelty claims require experimental support and comparison with CUP-Mem.

3 Proposed system and intervention

Integration. Smurfs will retain planning and execution. A replaceable adapter will receive task-scoped events and return source-linked evidence. We will use Jev's `observe()` after permitted observations, `add_summary()` for source-linked summaries, and `retrieve()` before evidence-dependent planning or execution. Smurfs will generate the task answer. Write-back via `promote()` is explicit; retrieval alone does not promote findings. The Smurfs verifier will receive relevant evidence as well as the candidate answer.

Evidence model. Immutable records will distinguish user statements, tool observations, agent inferences, and verifier opinions. Summaries and decisions will record their source IDs. Roles within one task group share memory; independent groups and experimental conditions remain isolated. One coordinator will serialize writes initially. Multiple roles do not imply parallel execution, and inter-agent agreement does not establish truth.

Candidate intervention. Begin with explicit, authoritative updates to identifiable constraints. Preserve old records, record supersession, mark dependent summaries for review, and check affected evidence before reuse. When budget permits, retrieve raw evidence and rebuild the affected conclusion; otherwise expose uncertainty. Temporal order alone will not resolve unrelated or ambiguous claims. Automatic implicit-conflict detection is an extension, not an assumed capability: STALE will also test the explicit-update mechanism's limitations.

4 Data and experimental protocol

Two primary memory evaluations. Use cleaned LongMemEval_S for general memory abilities (500 questions) [4] and STALE for state resolution, premise resistance, and policy adaptation (400 scenarios, three queries each) [10]. These are evaluation resources, not a training/test pair. Version 1 trains no model parameters; disjoint development data support prompt and threshold choices. STALE contains LongMemEval-derived distractors, so shared histories must be audited rather than treating both datasets as independent replications. Use the existing STALE release instead of regenerating a supposedly equivalent benchmark.

Action and failure study. Pilot MemoryArena travel planning using its existing tools and constraints [5]. Retain original tasks and separately create paired update/no-update variants with a recorded intervention point. These variants are an EviRecall stress suite, not official MemoryArena scores. MAST's 14-category taxonomy and annotated traces inform failure annotation [6]; they are not executable training tasks for Smurfs. If the action environment proves infeasible, explicitly limit conclusions to memory and controlled cross-role tests. MemoryAgentBench's conflict-resolution subset is a fallback, with source overlap checked [9].

Comparisons. At the memory layer, compare recent history, BM25 over identical raw chunks, unmodified Jev, and Jev with our intervention. Include CUP-Mem on a matched STALE subset if it passes the pilot; otherwise disclose the missing direct comparison and narrow claims. Fix the task-answer model and disclose additional component models. Action experiments compare Smurfs without persistent memory, with unmodified Jev, and with our intervention. Ablations disable validity filtering and raw-evidence recovery separately. Any claim that multiple agents help requires a budget-matched single-agent control.

Isolation and scale. Split by complete history or task group before selecting questions. Answers, evidence-location labels, conflict explanations, and environment scoring state remain evaluator-only. Reset stores between conditions and fix whether question-level write-back is allowed. LongMemEval S, M, and oracle are context settings, not splits. Start with 10 development units per primary dataset and 3 travel groups; provisionally target 100 held-out LongMemEval questions, 100 held-out STALE scenarios, and 20 travel groups, subject to measured costs and available disjoint groups. Freeze IDs, models, prompts, judge, and budgets before testing. Report subset coverage and three repetitions where affordable; estimate uncertainty at the history/scenario/group level.

5 Metrics and resource experiments

Use official memory scorers and task-success criteria, category-level results, and human checks. Distinguish citation existence, evidence recall, and semantic support. On the update suite, measure stale-use rate (stale-dependent decisions / eligible post-update decisions), cross-role propagation (update episodes with stale evidence reused by another role / update episodes), and recovery (corrected successful episodes / update episodes). Report counts and annotation disagreement; inspect successful and failed trajectories.

Record all role calls, memory writes, summaries, retrieval planning, Jev judgments, retries, timeouts, and terminal failures. Separate ingestion, online execution, and evaluation-judge costs; unavailable usage is unknown, not zero. Compare a small predeclared budget grid and concurrency of 1, 2, and 4, with fixed workload and retry policy. Report quality, tokens, request counts, throughput, and latency including queue time. Label injected delays as synthetic. API measurements cannot establish GPU utilization or hardware-scheduling performance. Avoid an unmanageable model-by-dataset-by-load factorial experiment.

6 Three work packages and collaboration

Member A: memory method and agent hooks. Own the adapter, minimal Smurfs read/write and verifier hooks, Jev baseline, update intervention, and mechanism ablations. Deliver a documented module and retrieval examples; write the method and ablation sections. This package favors algorithm work.

Member B: evidence and error experiments. Own the STALE/task data contract, travel-tool adaptation, paired update scenarios, evidence-support rubric, MAST-informed diagnosis, and recovery analysis. Deliver versioned scenarios and an independently usable trace scorer; write the task design and error-analysis sections. Begin with fixtures and recorded traces without waiting for A's module.

Member C: benchmark and resource measurement. Own the common runner, split/configuration manifests, memory-QA loading, cost/event collection, official-scorer wrappers, and budget/load experiments. Deliver reproducible commands and result aggregation; write the evaluation setup and efficiency sections. This package favors backend/AI infrastructure. C does not own all task environments or baseline implementations.

Each package includes implementation, experiments, analysis, and writing. Agree on a minimal memory/event/task interface first, integrate a small example in week 1, and cross-review another package. Merge individual sections around one protocol and result set; jointly edit the abstract, conclusions, terminology, and contribution statement. Independent ownership requires early integration rather than postponing integration until the report.

7 Milestones and expected deliverables

Week 1: interface fixtures; small Smurfs/Jev, STALE/CUP-Mem, and travel-environment feasibility checks; cost estimate and scope freeze. **Weeks 2-3:** baselines, one intervention, shared traces, and development-only refinement. **Week 4:** frozen tests, paired comparisons, ablations, and a small resource sweep. **Week 5:** cross-reproduction, report, poster, and demonstration. These are proposed stages, not official course deadlines.

Deliver a reusable memory integration, controlled evidence/error experiments, and a reproducible harness with positive or negative results. Artifacts will identify upstream versions, changes, configurations, splits, and limitations. Course deliverables include a LaTeX report, contribution statement, GitHub source, and poster presentation [16]. Names, provider access, spending limits, the action environment, and the submission template remain to be finalized. This proposal specifies the research design; implementation belongs to the three members.

References

[1] Junzhi Chen, Juhao Liang, Benyou Wang. Smurfs: Multi-Agent System using Context-Efficient DFSDT for Tool Planning. 2025. NAACL, pp. 3281-3298. [source]

[2] Qiming Zhu, Shunian Chen, Rui Yu et al.. From Lossy to Verified: A Provenance-Aware Tiered Memory for Agents. 2026. arXiv:2602.17913. [source]

[3] FreedomIntelligence. Jev TierMem: Lightweight Agent Memory. 2026. Software documentation; accessed 2026-10-27. [source]

[4] Di Wu, Hongwei Wang, Wenhao Yu et al.. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. 2025. arXiv:2410.10813. [source]

[5] Zexue He, Yu Wang, Churan Zhi et al.. MemoryArena: Benchmarking Agent Memory in Interdependent Multi-Session Agentic Tasks. 2026. arXiv:2602.16313. [source]

[6] Mert Cemri, Melissa Z. Pan, Shuyi Yang et al.. Why Do Multi-Agent LLM Systems Fail?. 2025. arXiv:2503.13657. [source]

[7] Yanki Margalit, Nurit Cohen-Inger, Erni Avram et al.. Governed Shared Memory for Multi-Agent LLM Systems. 2026. arXiv:2606.24535. [source]

[8] LangChain. Deep Agents Overview. 2026. Official software documentation; accessed 2026-10-27. [source]

[9] Yuanzhe Hu, Yu Wang, Julian McAuley. Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions. 2026. arXiv:2507.05257. [source]

[10] Hanxiang Chao, Yihan Bai, Rui Sheng et al.. STALE: Can LLM Agents Know When Their Memories Are No Longer Valid?. 2026. arXiv:2605.06527. [source]

[11] Adam Fourney, Gagan Bansal, Hussein Mozannar et al.. Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks. 2024. arXiv:2411.04468. [source]

[12] Yuzhe Yang, Yifei Zhang, Minghao Wu et al.. TwinMarket: A Scalable Behavioral and Social Simulation for Financial Markets. 2025. arXiv:2502.01506. [source]

[13] Bingquan Zhang, Xiaoxiao Liu, Yuchi Wang et al.. Human or LLM as Standardized Patients? A Comparative Study in Medical Education. 2025. arXiv:2511.14783. [source]

[14] Wujiang Xu, Zujie Liang, Kai Mei et al.. A-Mem: Agentic Memory for LLM Agents. 2025. arXiv:2502.12110. [source]

[15] Prateek Chhikara, Dev Khant, Saket Aryan et al.. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. 2025. arXiv:2504.19413. [source]

[16] NLP Course, CUHK-Shenzhen. CSC5051/CSC6052/MDS5110 Natural Language Processing, Fall 2026. 2026. Course website; accessed 2026-10-27. [source]